线程池

当有大量的任务需要被执行, 这时间就想到了使用多线程. 正要new Thread(), 调用start方法的时候, 又想到了幼儿园老师曾经教过的, 线程资源很昂贵呀, 频繁创建和销毁线程的开销很大.

那为什么线程的创建和销毁开销比较大?

Java的线程模式是1对1的, 也就是一个Java线程对应一个操作系统的内核级线程. 那创建一个Java线程就得通过系统调用让操作系统切换到内核态去创建内核级线程, 这里就涉及到了内核态的切换, 涉及到线程资源分配, 栈空间的分配. 同样的, 销毁线程也需要操作系统切换到内核态回收这些资源. 所以Java中线程创建和销毁的开销是很大的.

然后Java21出现了虚拟线程, 虚拟线程或者说协程, 它是多对多的线程模型, 基于用户态的调度, 所以很轻量级. 但是你看着项目里的Java8陷入了沉默.

线程池的执行过程

那既然Java8中线程创建和销毁开销比较大, 那有没有一种可能, 可以把创建好的线程放到一个容器中, 需要用的时候再去拿, 用完后也不销毁, 而是放回去. 这样每次复用容器中已有的线程, 就能避免线程频繁地创建销毁. 这就是我们幼儿园老师经常说的线程池.

那么问题来了, 一般情况下, 线程池中的线程也就几十个, 但是要处理的任务往往有成百上千个, 你把线程累死, 给cpu干冒烟了, 一下子也处理不完这么多任务呀. 那线程处理不完的任务怎么办?

- 排队等待. 搞一个阻塞队列, 把暂时处理不完的任务放阻塞队列中等着. 当线程处理完手头的任务后, 从阻塞队列中拿出任务进行处理.

那么问题又来了, 阻塞队列大小是有上限的, 阻塞队列满了咋办?

- 队列满了, 说明任务量很多, 当前系统压力很大, 我们可以再新建一些临时线程来紧急处理.

那么如果队列中的任务都执行完了, 临时线程空闲了, 是不是应该销毁呢?

- 队列都没任务了, 说明系统不忙了, 不忙了临时工肯定要销毁, 所以需要设置一下临时线程的最大空闲时间, 超过这个时间后就销毁.

那么问题又来了, 如果任务量就是很大, 阻塞队列满了, 临时线程也都在忙, 这时候又新来了任务应该怎么办呢?

- 这个时候就要定义一套拒绝策略来拒绝这些任务了.

话又说回来了, 这个线程池应该以什么方式创建线程呢, 应该创建什么样的线程呢, 普通线程还是守护线程, 那创建线程的时候应该怎么给线程命名呢?

- 所以最好搞一个线程工厂来统一负责创建线程
- 顺便可以根据具体业务给线程和线程池命名, 这样的话业务出问题, 就能定位到对应的线程池和线程. 线程出了问题也能定位到具体业务.
- 所以这也说明了不同的业务共用同一个线程池肯定是不好的. 不同业务应该使用不同的线程池

以上就是线程池的七大参数了

- 核心线程数, 最大线程数, 临时线程的空闲时间, 时间单位, 阻塞队列, 拒绝策略, 线程工厂.

- ```
  public ThreadPoolExecutor( @Range(from = 0, to = java.lang.Integer.MAX_VALUE) int corePoolSize,
                             @Range(from = 1, to = java.lang.Integer.MAX_VALUE) int maximumPoolSize,
                             @Range(from = 0, to = java.lang.Long.MAX_VALUE) long keepAliveTime,
                             @NotNull TimeUnit unit,
                             @NotNull BlockingQueue<Runnable> workQueue,
                             @NotNull ThreadFactory threadFactory,
                             @NotNull RejectedExecutionHandler handler) {
  ```
- 总结一下线程池的执行流程, 当一个任务被提交到线程池, 如果核心线程空闲, 那就交给核心线程执行. 否则尝试放入阻塞队列. 如果阻塞队列没满就入队, 如果满了, 就创建临时线程进行处理. 如果队列满了, 且临时线程都在忙, 那么就触发拒绝策略.

不想拒绝任务怎么办?

有大聪明就要说了, 可以把阻塞队列设置的大一点, 设置成无界队列, 这样的话来再多任务都能容纳, 就不用拒绝了. 队列可以是无界, 但是服务器内存有限啊, 随着任务越来越多, 内存肯定会满, 最后频繁gc, 然后触发oom.

那咋办呢? 有个拒绝策略叫CallerRunsPolicy, 就是谁往线程池中提交任务, 谁就去执行这个任务. 这种方式确实可以避免丢弃任务, 但是让业务线程去执行这些任务可能会导致业务线程被拖慢, 无法及时处理前端发的请求. 所以这个拒绝策略只能适用于低并发还不想丢弃任务的场景.

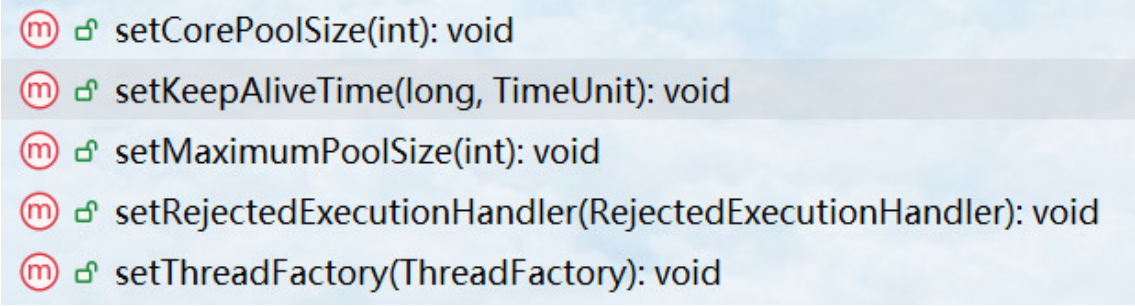
你又想性能好, 又不想丢弃任务, 那怎么办呢?

- 你想想操作系统中学到的内存和磁盘, 想要内存数据不会断电丢失就需要持久化到磁盘中.
- 再想想操作系统虚拟内存中的请求分页, 当无法加载所有数据到内存时, 只加载必要的, 需要其他数据时触发缺页中断, 然后从磁盘中读入内存.
- 所以线程池想要任务不被丢弃, 就做任务持久化. 那就自定义拒绝策略, 把要拒绝的任务存到mysql, 或者redis, 或者mq中.
  ```

- 然后重写线程池的afterExecute方法, 当线程池执行完一个任务时, 说明此事线程池有可能空闲了, 那么判断线程池中阻塞队列的大小, 如果阻塞队列还比较空, 那么读取mysql或者redis或者mq中持久化的任务, 放入线程池中再进行处理. 这就做到了不丢弃任务.
- 除了重写afterExecute方法之外, 还可以自定义阻塞队列, 当线程从阻塞队列中取任务时, 判断阻塞队列中任务量的多少, 如果阻塞队列任务量少, 则从mysql/redis/mq中读取数据, 放入队列中. 然后线程池就可以执行这些任务了. 这也做到了不丢弃任务.

动态参数线程池

动态参数的线程池怎么设计? 或者说怎么在不重启应用的情况下动态调整线程池核心参数?

- 线程池本身提供了一系列的set方法, 让你能够在运行时修改核心参数, 比如setCorePoolSize(), setMaxPoolSize()等等. 调用set方法就能动态修改核心线程数, 最大线程数等等.
- 

- 然后将配置放入配置中心, 线程池监听配置中心, 当配置中心配置变更, 线程池能监听到, 然后动态修改.
- 但是有一个问题, Java中的阻塞队列的容量大小是final修饰的, 也就是说阻塞队列大小不可变. 所以这里要想实现阻塞队列大小可变, 就得自己重写一个大小可变的阻塞队列.

- LinkedBlockingQueue.java

```java
public class LinkedBlockingQueue<E> extends AbstractQueue<E>

    The capacity bound, or Integer.MAX_VALUE if none
    private final int capacity;
```

- 这样就实现了不重启应用, 修改配置中心的配置, 即可动态调整线程池核心参数.

线程池状态监控

怎么监控线程池状态? 或者说怎么自己设计一个简单的线程池监控呢?

- 第一步, 你得定义一系列线程池指标呀. 比如活跃线程数, 线程池当前线程数, 阻塞队列大小, 已完成任务数, 总任务数等等.
- 第二步, 得收集这些指标呀, 怎么收集呢. 可以使用定时任务, 每隔几秒采集一下这些指标数据. 也可以重写afterExecute, 或者beforeExecute方法, 在执行前后或者任务提交时去采集这些指标数据.
- 第三步把这些指标数据上报到监控系统中
- 第四步, 通过可视化图表的方式来展示这些指标数据.